

"Good microservices design starts with business boundaries — not REST controllers."

MODULE 2

Microservices Design

From business domains to well-designed, independently evolving services.

**Service
boundaries**

**Loose coupling &
high cohesion**

**RESTful API
design**

**Integration
patterns**

**Service
communication**

**Stateless vs.
stateful**

LEARNING OBJECTIVES

Define boundaries, design REST APIs, choose communication and understand state.



BY THE END OF THIS MODULE, YOU CAN

1. Identify appropriate service boundaries
2. Apply loose coupling and high cohesion
3. Explain bounded contexts
4. Design RESTful APIs
5. Compare API and messaging integration
6. Select communication strategies
7. Explain stateless and stateful services

WHY IT MATTERS These answers decide whether the architecture really supports independent deployment and scaling.

KEY TAKEAWAY Boundaries, communication and state decide whether services can really deploy and scale independently.



THREE MAJOR DESIGN QUESTIONS

1

Where should we create service boundaries?

Should Payment be part of Order Service — or its own service?

2

How should services communicate?

REST? gRPC? Messaging? Events?

3

Where should state live?

With several Order Service instances, can any instance handle the next request?

FROM MODULE 1 TO MODULE 2

Principles are in place — now we design.

MODULE 1 GAVE US

- ✓ Why monoliths struggle to change and scale
- ✓ What makes a service a microservice
- ✓ Data ownership and API / event communication
- ✓ The cost of distribution and designing for failure
- ✓ Twelve-Factor and DevOps habits

NOW Turn those principles into concrete design decisions.



THE SEVEN-STEP DESIGN PATH

- 1 Understand the business requirement **Part 1**
- 2 Identify business capabilities **Part 1**
- 3 Define the domain and bounded contexts **Part 1**
- 4 Identify service boundaries **Part 1**
- 5 Design responsibilities and API contracts **Part 2**
- 6 Choose communication and state strategy **Parts 3–4**
- 7 Validate and refine the design **Exercise 1.2**

KEY TAKEAWAY Design moves from business requirements to boundaries, contracts, communication and state — in that order.

PART 1: FINDING SERVICE BOUNDARIES

Boundaries decide almost everything that follows.



SO FAR

- ✓ Microservices own business capabilities
- ✓ They deploy, scale and store data independently
- ✓ Independence costs network calls and failure handling

NEXT QUESTION

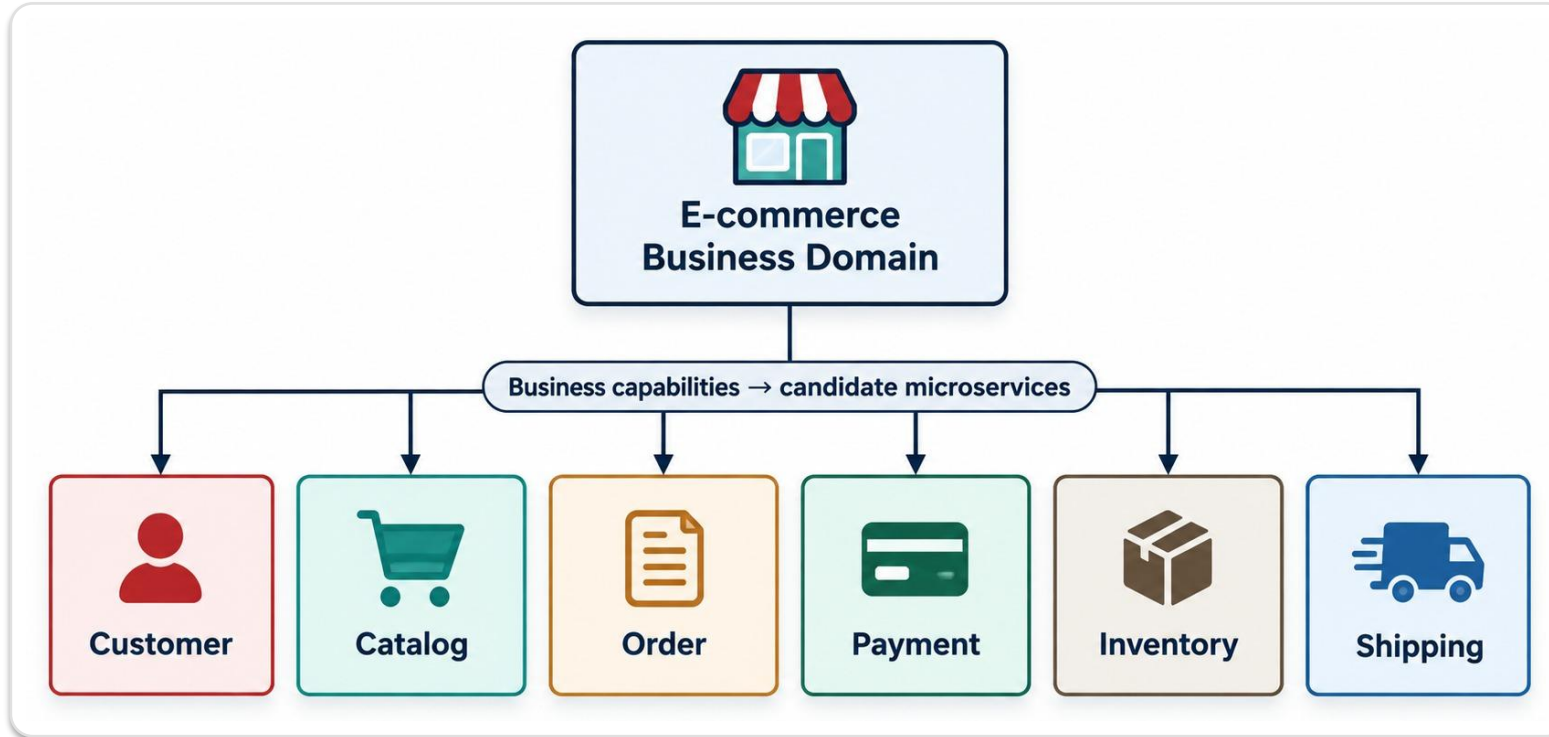
Where exactly should one service end and the next begin?

IN THIS PART

- 1 From business domain to candidate services
- 2 What makes a good boundary — and the right size
- 3 Coupling and cohesion
- 4 Domain-Driven Design and bounded contexts
- 5 Decomposition, with an exercise
- 6 Migrating safely and avoiding the distributed monolith

FROM BUSINESS DOMAIN TO MICROSERVICES

How business capabilities become candidate services.



What the business does

Manage customers and products, create orders, take payments, manage stock and ship.

Capabilities → candidates

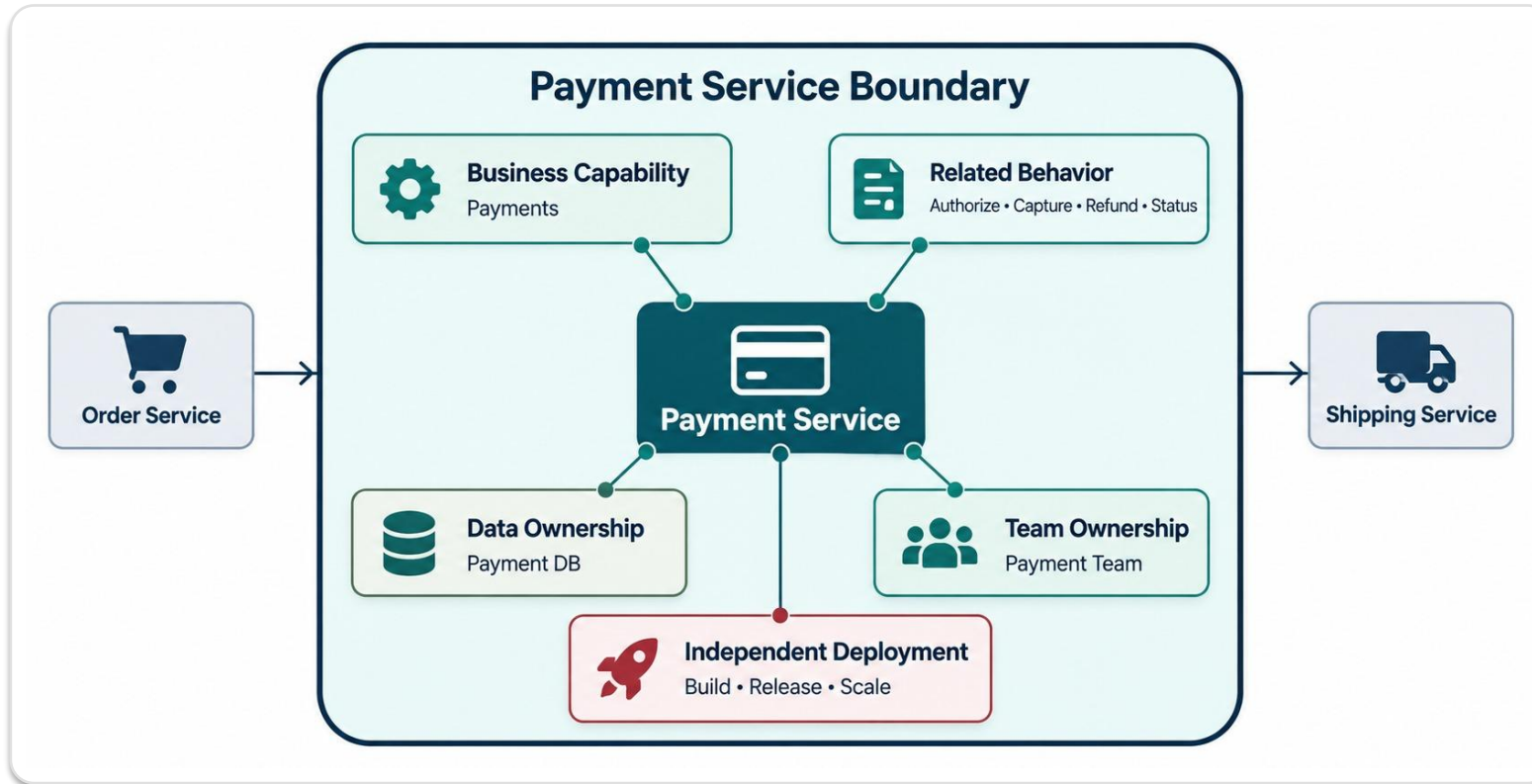
Order Management → Order Service; Payment Processing → Payment Service.

CANDIDATES, NOT FINAL Domain analysis, teams and change patterns decide.

KEY TAKEAWAY Start with the business domain — not with “how many services should we create?”

WHAT MAKES A GOOD MICROSERVICE BOUNDARY?

Business ownership, data ownership, change patterns and autonomy.



PAYMENT SERVICE: GOOD FIT

- ✓ Authorize payment
- ✓ Capture payment
- ✓ Refund payment
- ✓ Payment status

Doesn't belong

Update shipping address → Shipping, not Payment.

ASK What changes together? It belongs together.

KEY TAKEAWAY Things that change together belong together; things that must evolve independently may need separate boundaries.

RIGHT-SIZING A SERVICE

Not too fine, not too coarse — sized to one capability and one team.

TOO FINE-GRAINED

- Chatty network calls
- High operational overhead
- One table = one service

RIGHT-SIZED

- ✓ One business capability
- ✓ Owned by one team
- ✓ Deploys independently

TOO COARSE-GRAINED

- Many unrelated capabilities
- Teams block each other
- A distributed monolith

ASK FOR EVERY SERVICE

- ? Which business capability does it own?
- ? Which data does it own?
- ? Which team owns it?
- ? How often does it change — and scale?

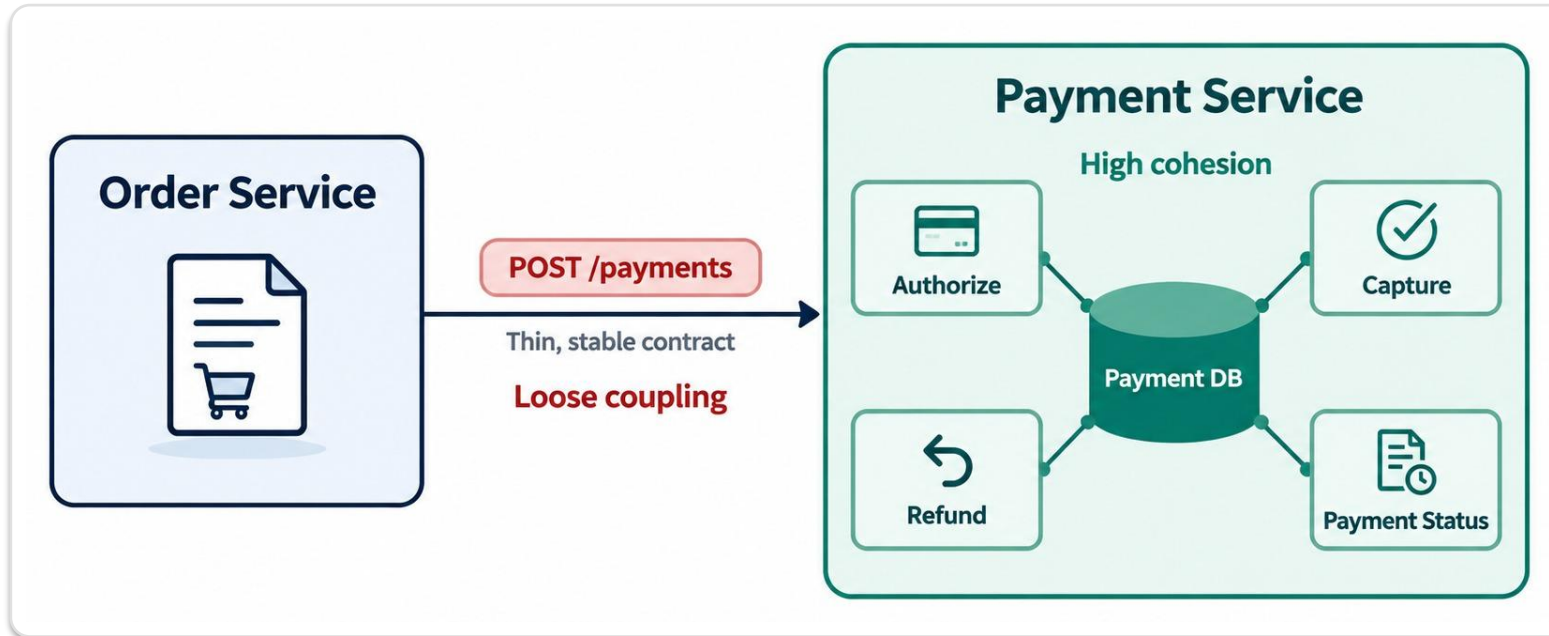
WARNING SIGNS OF POOR BOUNDARIES

- ✗ One feature needs changes in many services
- ✗ Services share tables
- ✗ Services can't deploy on their own
- ✗ The same business rule lives in several services

KEY TAKEAWAY Right-sized services balance independent deployment against the cost of running many small services.

LOOSE COUPLING AND HIGH COHESION

Two fundamental characteristics of well-designed services.



High cohesion — inside

Authorize, capture, refund and status all live in Payment Service.

ORDER DOESN'T NEED TO KNOW

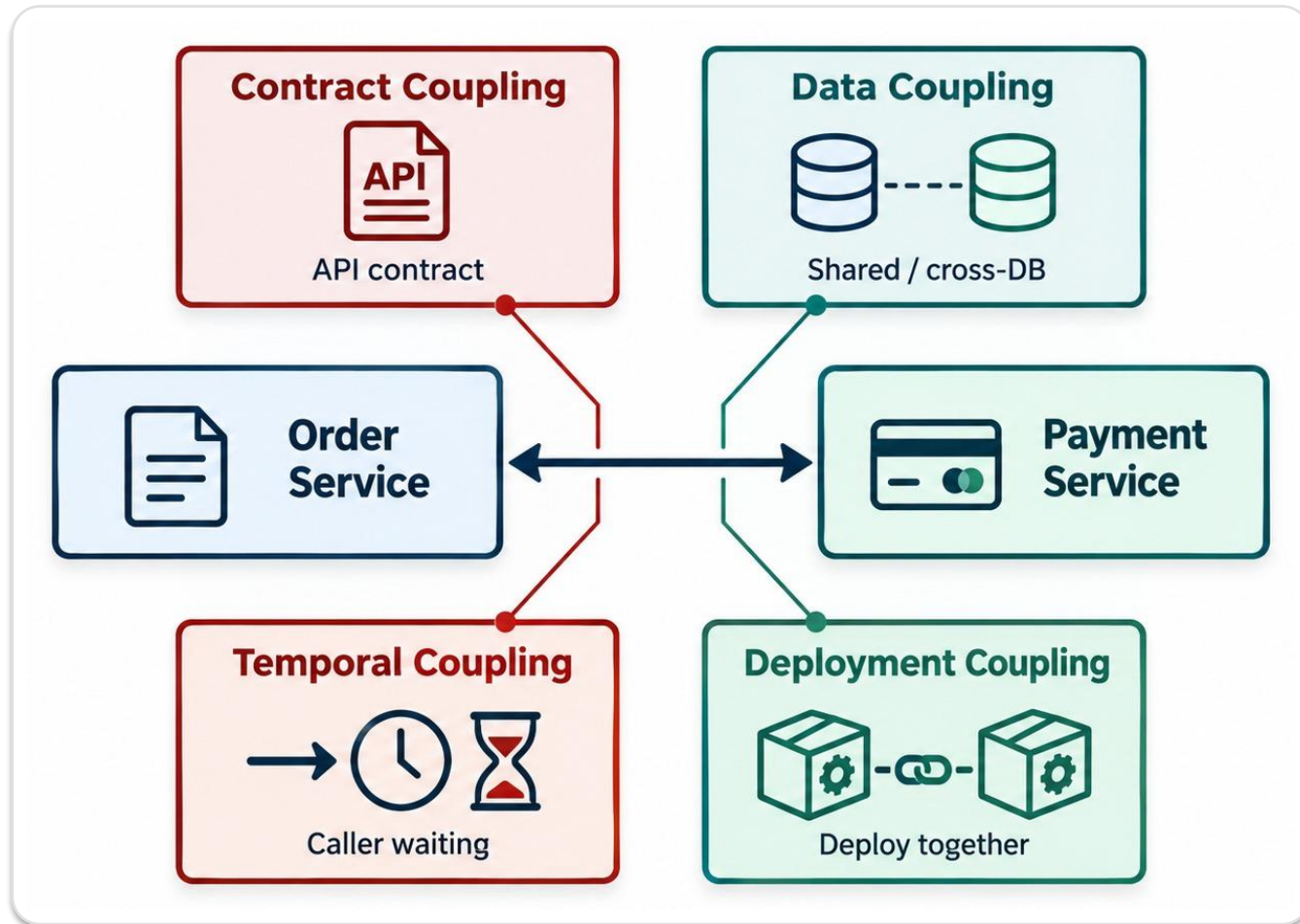
- ✗ Payment's tables
- ✗ Its Java classes
- ✗ Its internal algorithms

RULE Cohesive inside, loosely coupled between.

KEY TAKEAWAY Keep related payment behavior together, and let other services depend only on its contract.

UNDERSTANDING COUPLING BETWEEN SERVICES

Contract, data, temporal and deployment coupling.



Contract

Depends on another service's API shape.

Data

Several services depend on one schema.

Temporal

Service B must be up when A calls.

Deployment

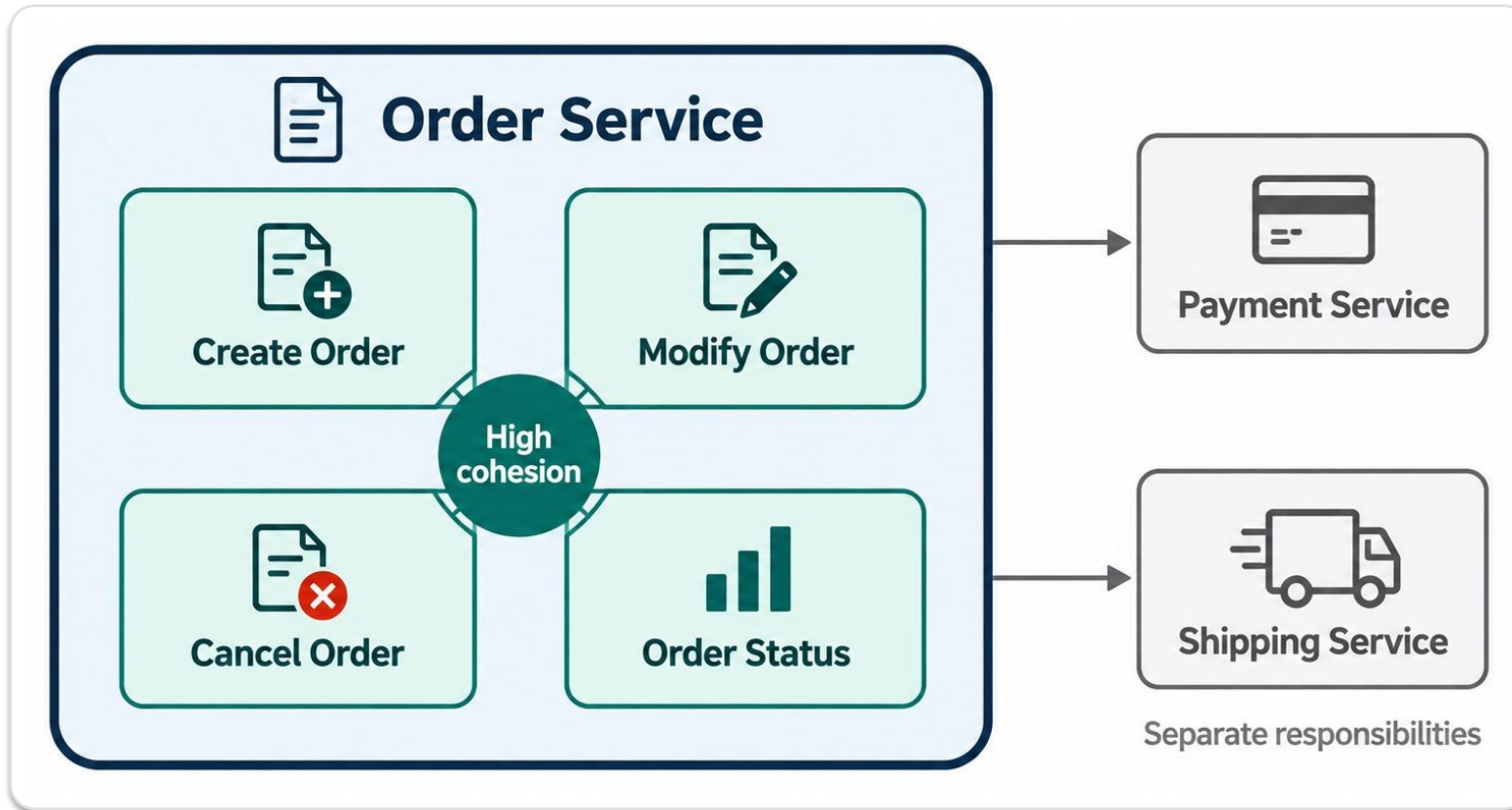
Changing A forces B to redeploy.

THE GOAL Intentional, manageable coupling
— not zero.

KEY TAKEAWAY Know which kind of coupling you are creating — and keep it intentional and manageable.

HIGH COHESION WITHIN A SERVICE

Keep closely related business behavior and data together.



Order Service owns

Create, modify, cancel and track orders.

Shared business logic

Pricing or eligibility rules in a shared library force services to release together.

Safe to share

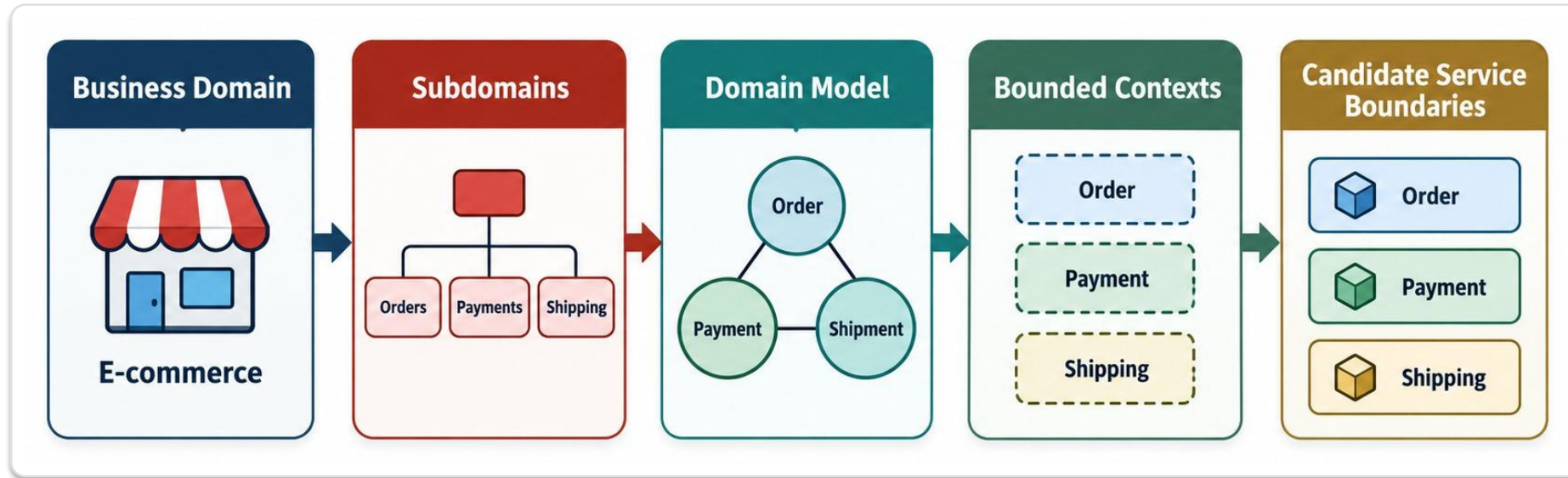
Technical utilities: logging, tracing, HTTP client wrappers.

QUICK TEST Describe the service in one short sentence.

KEY TAKEAWAY A service should have one clear business purpose.

INTRODUCTION TO DOMAIN-DRIVEN DESIGN

Domain, subdomains, domain model, bounded contexts and candidate services.



Domain

The business area — here, e-commerce.

Subdomains

Core: invest most. Supporting: build simply. Generic: buy or reuse.

Ubiquitous language

One vocabulary shared by developers and business experts.

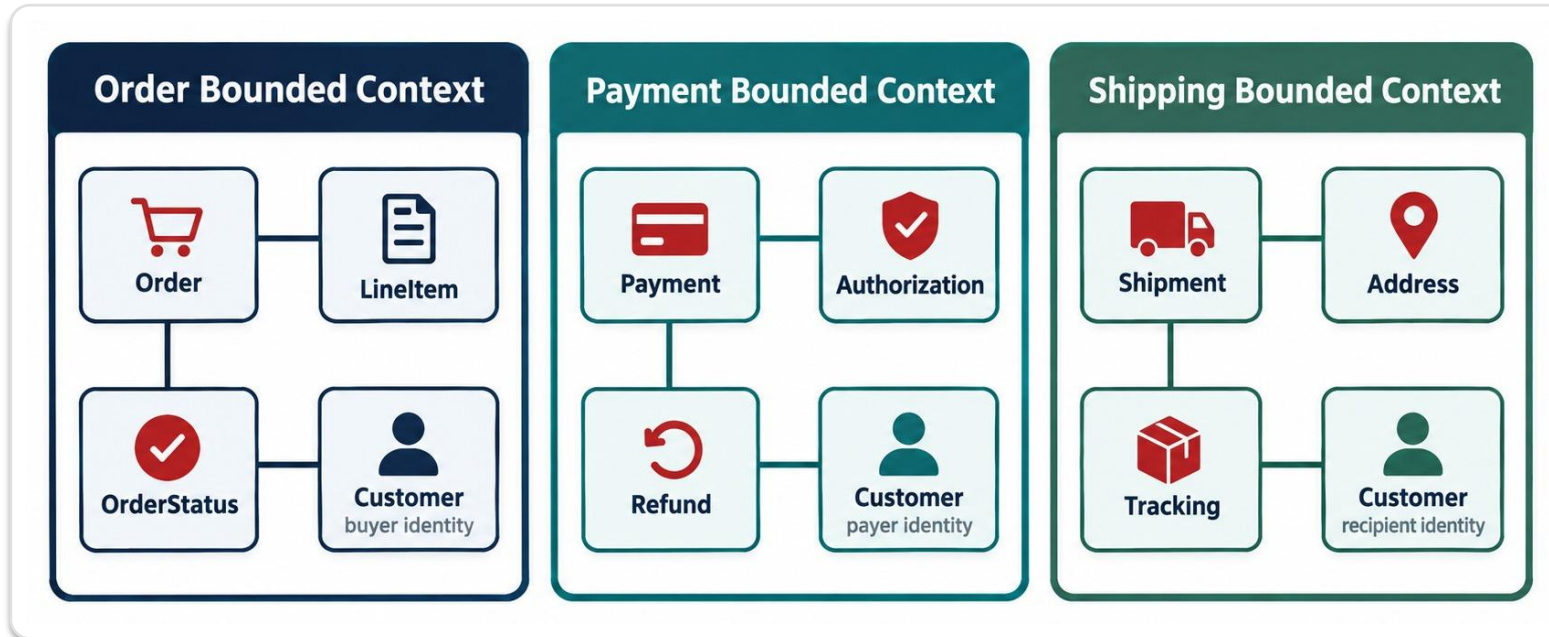
Bounded context

Where a model and its terms have one meaning.

KEY TAKEAWAY Use just enough DDD to find good service boundaries — starting from a shared business language.

BOUNDED CONTEXTS

Clear model and responsibility boundaries.



Same word, different model

"Customer" is an ID and shipping address in Ordering, but name, email and preferences in Customer Management.

Own concepts

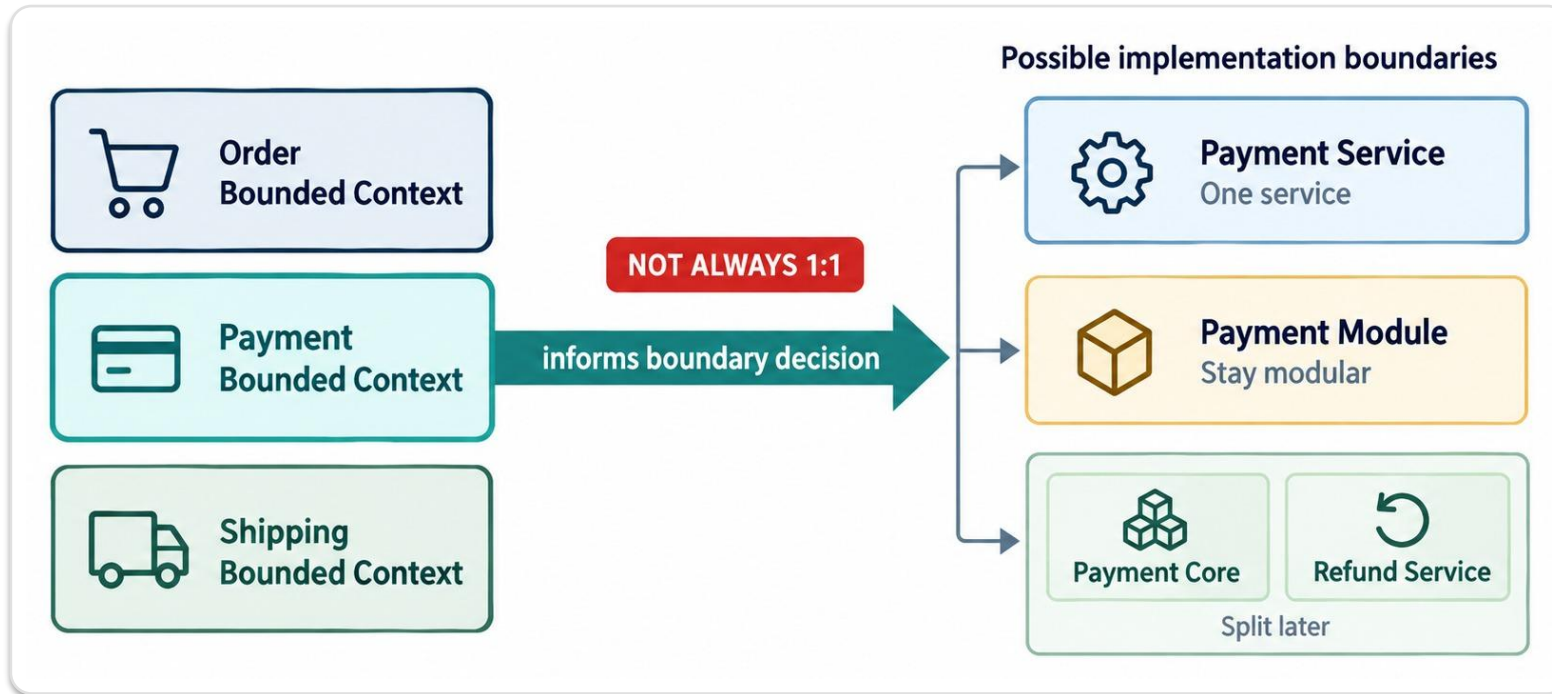
Order: line items, status. Payment: refunds, authorizations.

BIG SHIFT No single enterprise-wide object model.

KEY TAKEAWAY Inside each bounded context, the model and its terms have one clear meaning.

BOUNDED CONTEXT → SERVICE BOUNDARY

Contexts inform candidate services — without a strict 1:1 rule.



Bounded context

A domain modeling boundary.

Microservice

An architecture and deployment boundary.

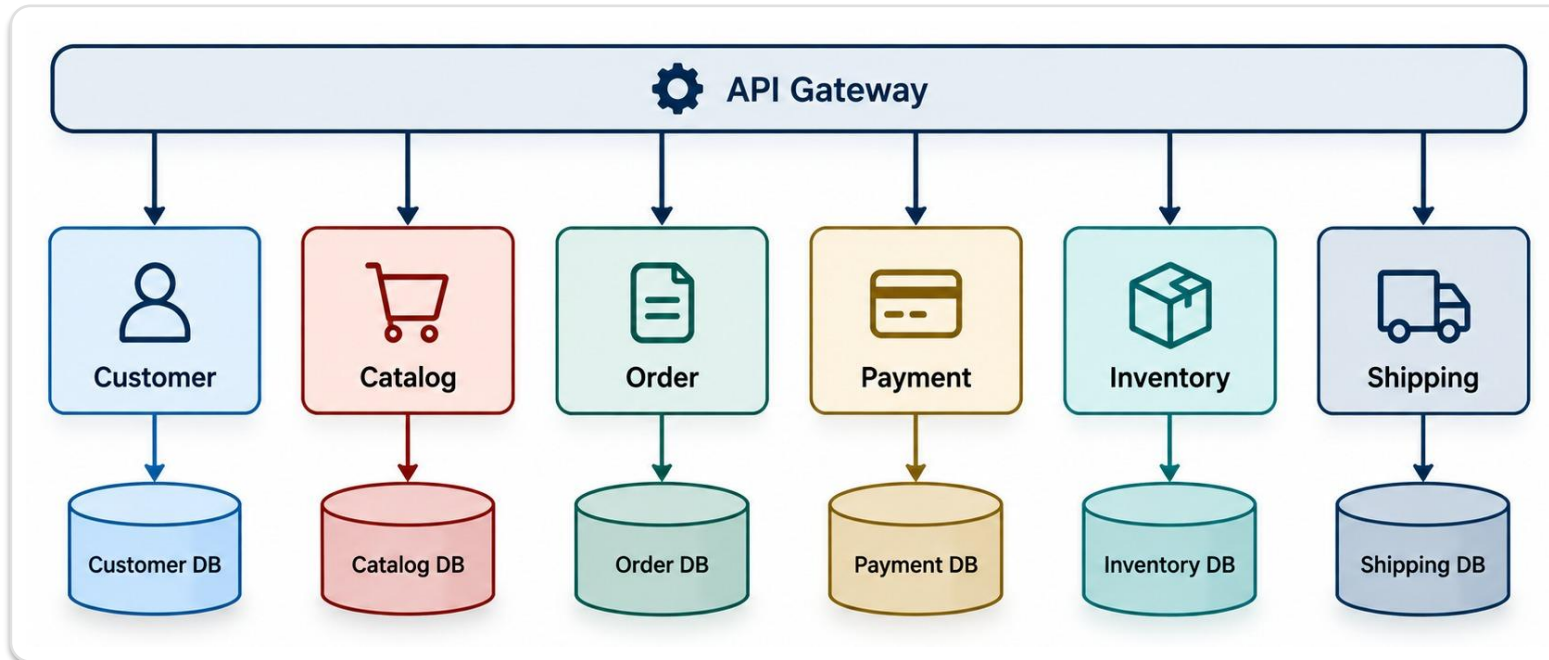
POSSIBLE OUTCOMES

- ✓ Often: one context → one service
- ✓ Large context → several services
- ✓ Or stay a module for now

KEY TAKEAWAY Let the business model inform the architecture — not blindly dictate the number of deployments.

SERVICE DECOMPOSITION WITH OWNED DATA

Six capabilities behind one gateway, each with its own database.



Customer · Catalog

Profiles and preferences; product information.

Order · Payment

Order lifecycle; authorize, capture and refund.

Inventory · Shipping

Stock and reservations; shipments and tracking.

KEY TAKEAWAY Breaking down the business produces candidate services with clear responsibilities.

EXERCISE: GROUP FEATURES INTO SERVICES

Practice bounded contexts and high cohesion on the order platform.

PRACTICE EXERCISE

10 MIN

SCENARIO

The order platform needs these features:

open a customer account · view a product ·
add to cart · place an order · charge a card ·
refund a payment · reserve stock · print a
shipping label · email a receipt · flag a
suspicious order

YOUR TASKS

- 1 Group the features into services
- 2 Name the data each service owns
- 3 Pick one feature that could go either way — and decide

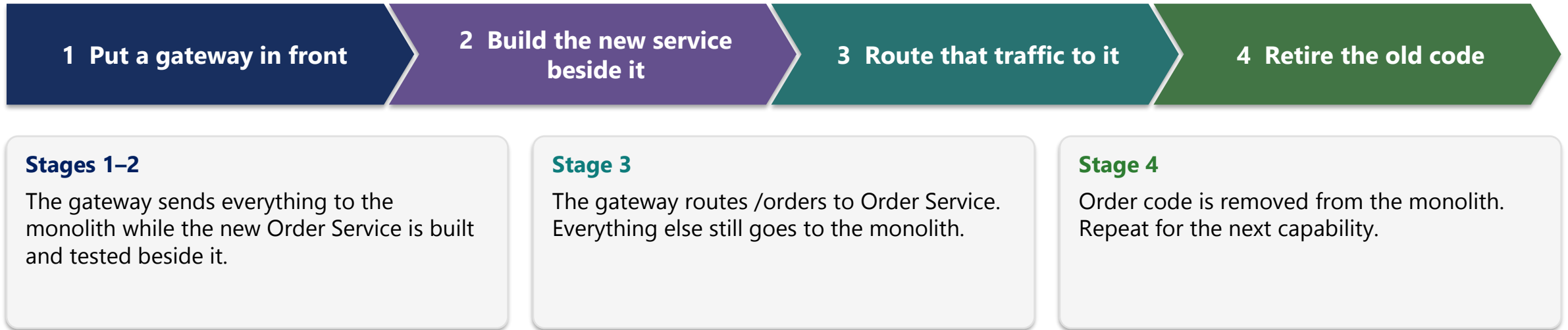
EXPECTED OUTCOME

- ✓ Customer, Catalog, Order, Payment, Inventory, Shipping
- ✓ Notification owns receipt emails
- ✓ Fraud checks: separate if they change often
- ✓ Every service owns its own data

KEY TAKEAWAY Group features by business capability and the data they change together.

DECOMPOSING A MONOLITH: THE STRANGLER FIG

Replace a monolith one capability at a time — safely.



WHY IT WORKS

- ✓ The monolith keeps serving traffic the whole time
- ✓ Every step is small and reversible
- ✓ Each capability is validated on real traffic

Where to start

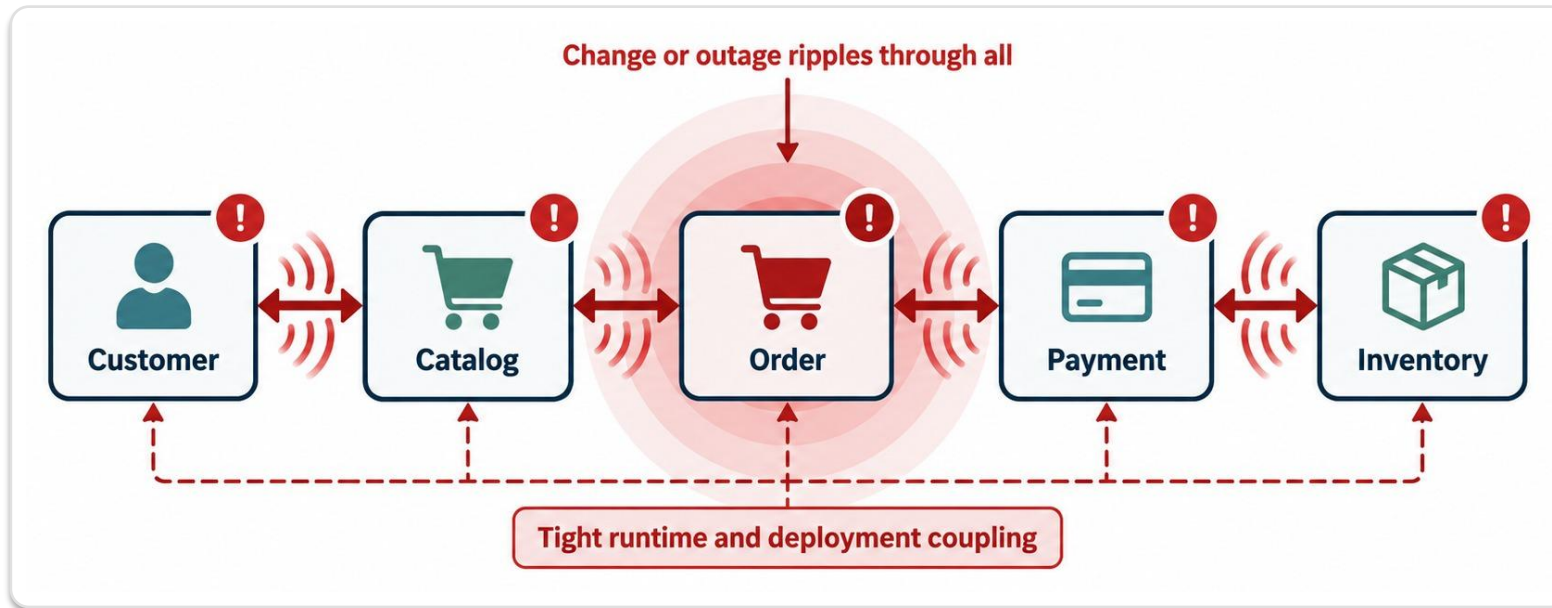
A low-risk, high-value seam with clear data ownership — in the lab, Account Service.

AVOID A big-bang rewrite that switches everything at once.

KEY TAKEAWAY Migrate one capability at a time while the monolith keeps running — never all at once.

AVOIDING THE DISTRIBUTED MONOLITH

How excessive dependencies defeat microservice independence.



WARNING SIGNS

- ! Services must deploy together
- ! Excessive synchronous calls
- ! Shared database tables
- ! Services know each other's internals

LOOKS LIKE MICROSERVICES ...but behaves like one tightly coupled system.

KEY TAKEAWAY Microservices are about independent evolution — not simply separate processes.

EXERCISE: SPOT THE TIGHT COUPLING

Mark each design good or bad — and fix the bad ones.

PRACTICE EXERCISE

10 MIN

SCENARIO

1. Order Service updates the Inventory database directly.
2. Order Service calls Inventory's API to reserve stock.
3. Notification polls the Order database every 5 seconds.
4. Order publishes OrderPlaced; Notification reacts.
5. Order and Payment share a pricing-rules library and must release together.

YOUR TASKS

- 1 Mark each design good or bad
- 2 Name the type of coupling for each bad one
- 3 Suggest a better design

EXPECTED OUTCOME

- ✓ 1 Bad: data coupling → call the API
- ✓ 2 Good: contract only
- ✓ 3 Bad: data coupling → consume events
- ✓ 4 Good: loose, asynchronous
- ✓ 5 Bad: deployment coupling → rules stay with the owner

KEY TAKEAWAY Services should never own each other's databases or internal logic.

PART 2: DESIGNING REST APIS

The contract is what other teams depend on.



SO FAR

- ✓ Boundaries follow business capabilities
- ✓ Coupling is intentional; cohesion is high
- ✓ Bounded contexts guide where services split

NEXT QUESTION

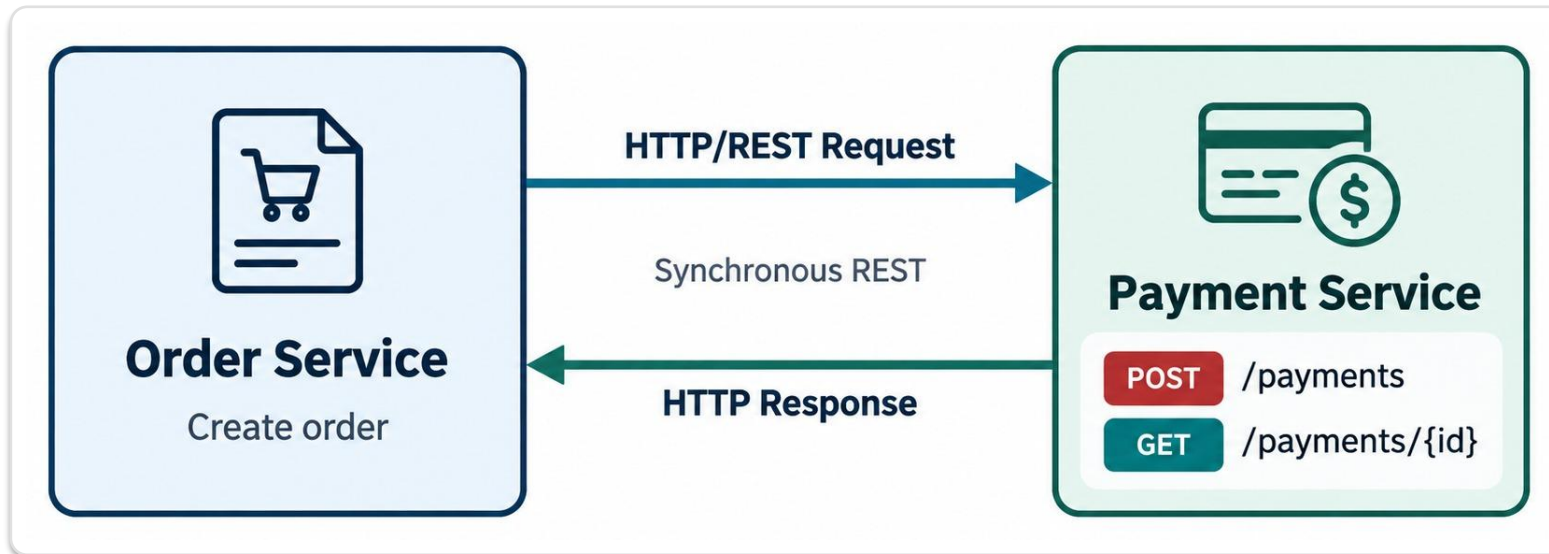
What should the contract between a service and its callers look like?

IN THIS PART

- 1 REST in microservices
- 2 Resources and URIs
- 3 HTTP methods and idempotency
- 4 Status codes and a Spring Boot controller
- 5 DTOs, validation and errors
- 6 Versioning, OpenAPI and API security

RESTFUL APIS IN MICROSERVICES

REST as a synchronous service communication mechanism.



REST BUILDING BLOCKS

Resources

HTTP methods

Representations

Stateless requests

Payment Service contract

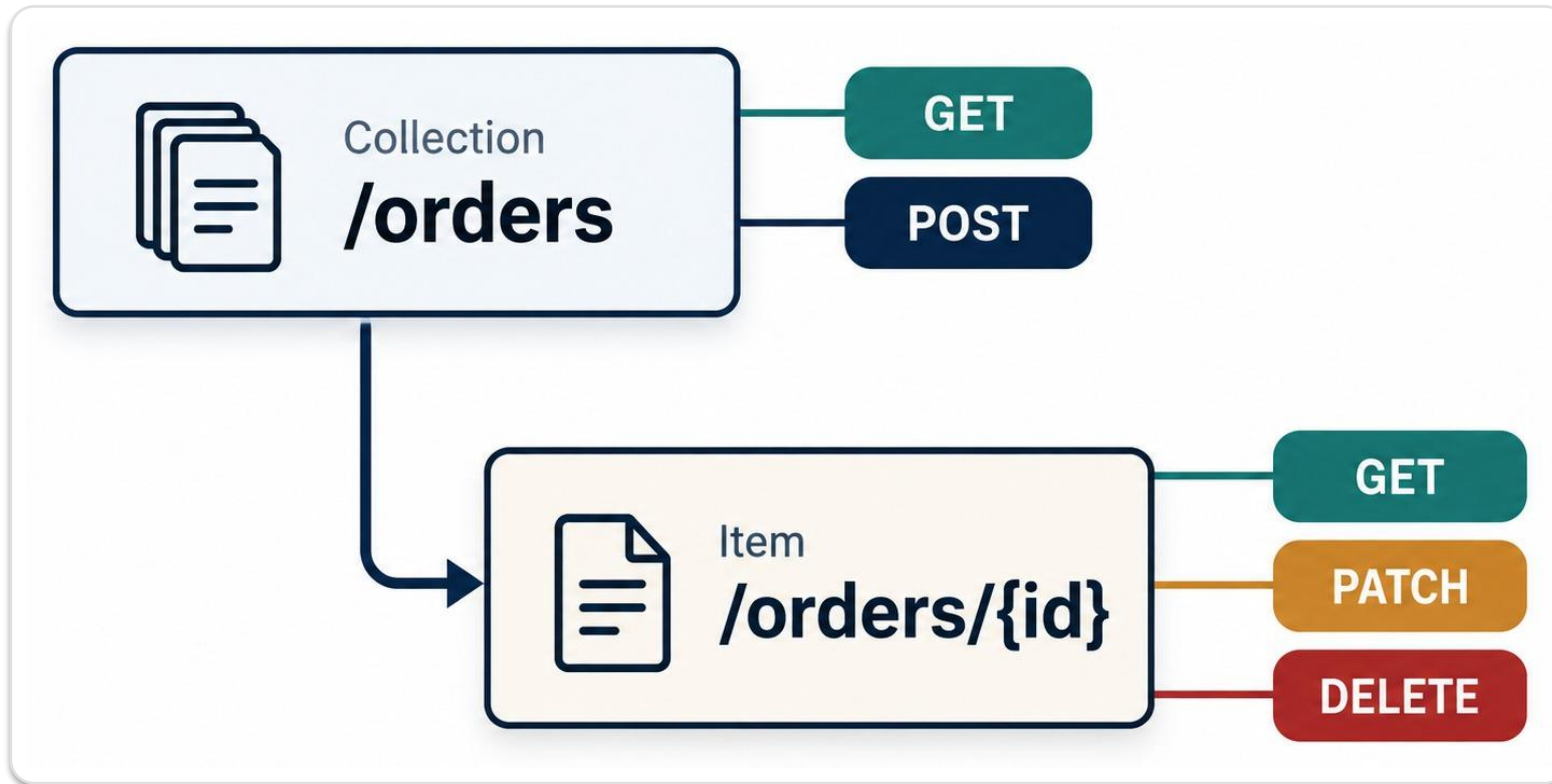
POST /payments
GET /payments/{id}

REST $\neq$ MICROSERVICES Neither implies the other.

KEY TAKEAWAY REST is one common mechanism for synchronous service communication.

RESOURCE-ORIENTED API DESIGN

Resources, URLs and nouns: /orders, /customers, /products.



AVOID RPC-STYLE URLS

- ✗ /getAllOrders
- ✗ /createNewOrder
- ✗ /deleteOrderById

Read it as

GET /orders/123 — retrieve the representation of order 123.

BE PRAGMATIC Action endpoints can fit; resources are the default.

KEY TAKEAWAY Name resources with nouns and let the HTTP method express the action.

HTTP METHODS AND CRUD SEMANTICS

GET, POST, PUT, PATCH and DELETE — used with the correct intent.

HTTP Method	Typical Purpose	Example
GET	Retrieve	GET /orders/123
POST	Create / process	POST /orders
PUT	Replace	PUT /customers/123
PATCH	Partial update	PATCH /orders/123
DELETE	Remove	DELETE /orders/123

GET should not change application state as its intended effect.

IDEMPOTENCY

PUT /customers/123 #1

PUT /customers/123 #2



Repeating the same PUT has the same intended effect as doing it once.

This matters when retries happen in distributed systems.

KEY TAKEAWAY Use each HTTP method for its intended meaning — idempotency matters once retries happen.

SAFE OPERATIONS AND IDEMPOTENCY KEYS

Which requests can be retried safely — and how to make POST retry-safe.

Method	Safe?	Idempotent?
GET	Yes	Yes
PUT	No	Yes
DELETE	No	Yes
PATCH	No	Not by default
POST	No	No



Retry-safe order creation

POST /orders
Idempotency-Key: 7f3a-checkout-001

- 201 Created (first request)
- 200 OK, same order (retry)

How it works

The server stores the key with the result and returns the original response for repeats.

WHY A double-tap must not create two orders or two charges.

KEY TAKEAWAY Safe methods never change state; idempotency keys make retries of POST safe.

HTTP STATUS CODES AND API RESPONSES

2xx, 4xx and 5xx — with consistent response and error design.

2XX SUCCESS

- 200** OK
- 201** Created
- 204** No Content

4XX CLIENT / REQUEST PROBLEM

- 400** Bad Request
- 401** Unauthorized
- 403** Forbidden
- 404** Not Found
- 409** Conflict

5XX SERVER-SIDE FAILURE

- 500** Internal Server Error
- 503** Service Unavailable

CONSISTENT ERROR RESPONSE

Error code

Message

Timestamp

Correlation / trace ID

Relevant details

Clients shouldn't reverse-engineer failures from arbitrary strings.

EXAMPLES

Order created
201 Created

Order doesn't exist
404 Not Found

Invalid input
400 Bad Request

KEY TAKEAWAY Status codes and a consistent error model are part of the API contract.

A SPRING BOOT REST CONTROLLER

Order Service exposes /orders — and the controller stays thin.

OrderController.java

```
@RestController
@RequestMapping("/orders")
public class OrderController {
    private final OrderService orders; // injected

    @GetMapping("/{id}")
    public OrderResponse get(@PathVariable String id) {
        return orders.find(id);
    }

    @PostMapping
    public ResponseEntity<OrderResponse> create(
        @Valid @RequestBody CreateOrderRequest req) {
        OrderResponse created = orders.create(req);
        return ResponseEntity
            .created(URI.create("/orders/" + created.id()))
            .body(created);
    }
}
```

Resource URL

@RequestMapping("/orders") names the resource.

GET by id

@GetMapping("/{id}") reads one order.

201 + Location

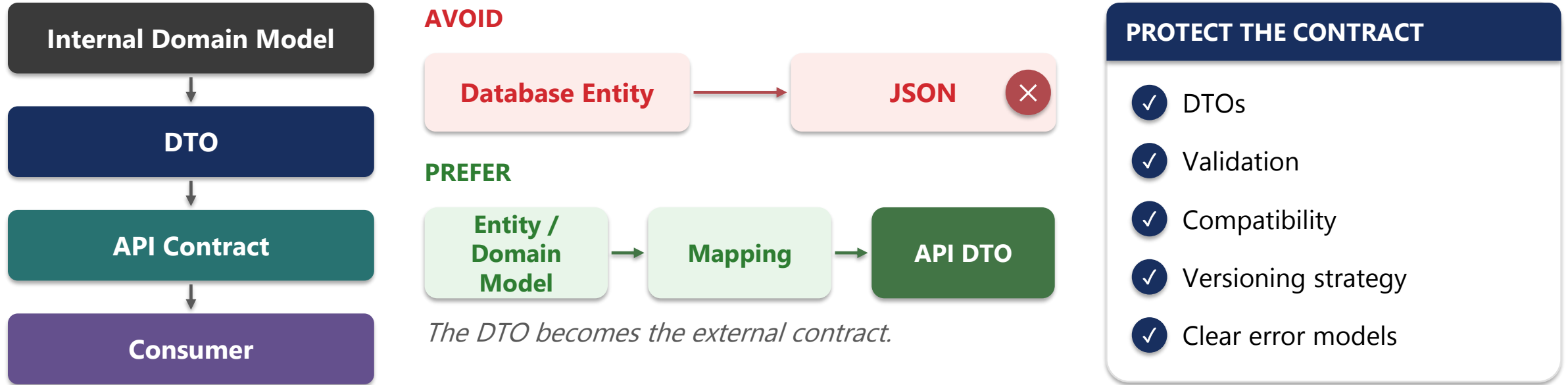
POST returns 201 Created and where to find the new order.

THIN CONTROLLER Business rules live in OrderService.

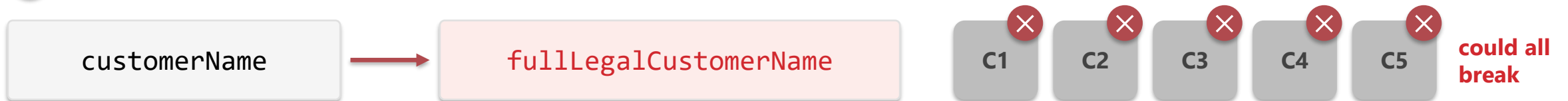
KEY TAKEAWAY Controllers map HTTP to the service layer — URLs, methods and status codes, nothing more.

DESIGNING STABLE MICROSERVICE API CONTRACTS

DTOs, validation, compatibility, versioning — and no implementation leakage.



⚡ BACKWARD COMPATIBILITY: ONE RENAME, FIVE CONSUMERS



KEY TAKEAWAY The DTO is the external contract — evolve it carefully so consumers keep working.

DTOS, VALIDATION AND CONSISTENT ERRORS

The request, the response and the error body are all part of the contract.

Request DTO with validation

```
public record CreateOrderRequest(
    @NotBlank String customerId,
    @NotEmpty List<@Valid Item> items,
    @NotBlank String currency) {}

public record Item(
    @NotBlank String sku,
    @Positive int quantity) {}
```

Validate at the edge

@Valid rejects bad input with 400 before business logic runs.

✓ Response DTO — no entity leaks

```
public record OrderResponse(
    String id,
    String status,
    BigDecimal total,
    Instant createdAt) {}
```

Hide internals

Responses expose only what clients need — never JPA entities.

✓ 400 error body — one shape

```
{
  "timestamp": "2026-09-14T10:15Z",
  "status": 400,
  "errorCode": "VALIDATION_FAILED",
  "message": "items must not be empty",
  "path": "/orders",
  "correlationId": "a1b2c3d4"
}
```

Traceable errors

A correlation ID links the error to logs across services.

KEY TAKEAWAY DTOs, validation and one error format keep the contract stable and easy to use.

API VERSIONING AND OPENAPI

Evolve contracts without breaking consumers — and publish them.

SAFE — BACKWARD COMPATIBLE

- ✓ Add an optional field
- ✓ Add a new endpoint
- ✓ Add an enum value clients can ignore

BREAKING — NEEDS A NEW VERSION

- ✗ Remove or rename a field
- ✗ Change a field's type or meaning
- ✗ Make an optional field required

RULE Version only breaking changes, and announce deprecation dates.

OpenAPI — the contract, kept with the code

```
openapi: 3.0.3
info:
  title: Order API
  version: 1.2.0
paths:
  /v1/orders/{id}:
    get:
      summary: Get one order
      responses:
        '200': { description: The order }
        '404': { description: Not found }
```

VERSIONING STYLES

/v1/orders

?version=2

Accept-Version: 2

Media type

KEY TAKEAWAY Evolve APIs with backward-compatible changes, version the breaking ones, and publish the contract as OpenAPI.

REST API SECURITY AND ANTI-PATTERNS

Protect every call — and fix the design mistakes teams repeat.

SECURE BY DESIGN

- ✓ Authenticate every request
- ✓ Authorize per resource — no guessable IDs
- ✓ Validate all input, headers and parameters
- ✓ HTTPS everywhere (TLS 1.2+)
- ✓ Return only the data clients need
- ✓ Rate-limit to prevent abuse

NEVER Return stack traces, SQL or secrets in an error response.

ANTI-PATTERNS TO FIX

- ✗ Verbs in URLs: /getOrders
- ✗ Always returning 200 OK
- ✗ Exposing database structure
- ✗ No pagination on large collections
- ✗ GET requests that change data
- ✗ Inconsistent names and error formats

Paginate, filter and sort large lists

```
GET /orders?status=PAID&page=2&size=20
    &sort=createdAt,desc
```

KEY TAKEAWAY Authenticate, authorize and validate every call — and fix each anti-pattern with a resource-oriented design.

PART 3: HOW SERVICES COMMUNICATE

A great API is only half the story — how we call it matters too.



SO FAR

- ✓ Resources, methods and status codes form the contract
- ✓ DTOs, validation and errors keep it stable
- ✓ Idempotency makes retries safe

NEXT QUESTION

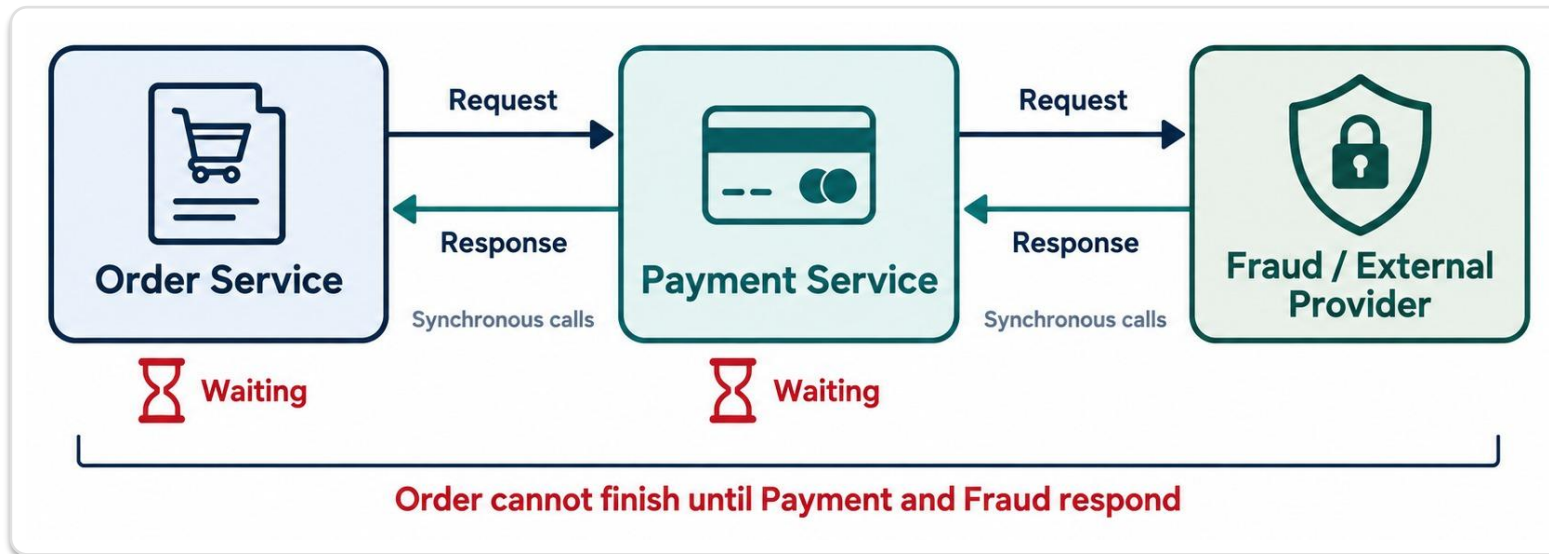
Should services call each other directly — or react to events?

IN THIS PART

- 1 Synchronous communication
- 2 Asynchronous messaging and events
- 3 API versus messaging — with code
- 4 Communication strategies
- 5 Gateway, composition, BFF and choreography
- 6 Avoiding chatty communication

SYNCHRONOUS SERVICE COMMUNICATION

Request/response with REST or gRPC — and the dependencies it creates.



BENEFITS

- ✓ Simple mental model
- ✓ Immediate result

CHALLENGES

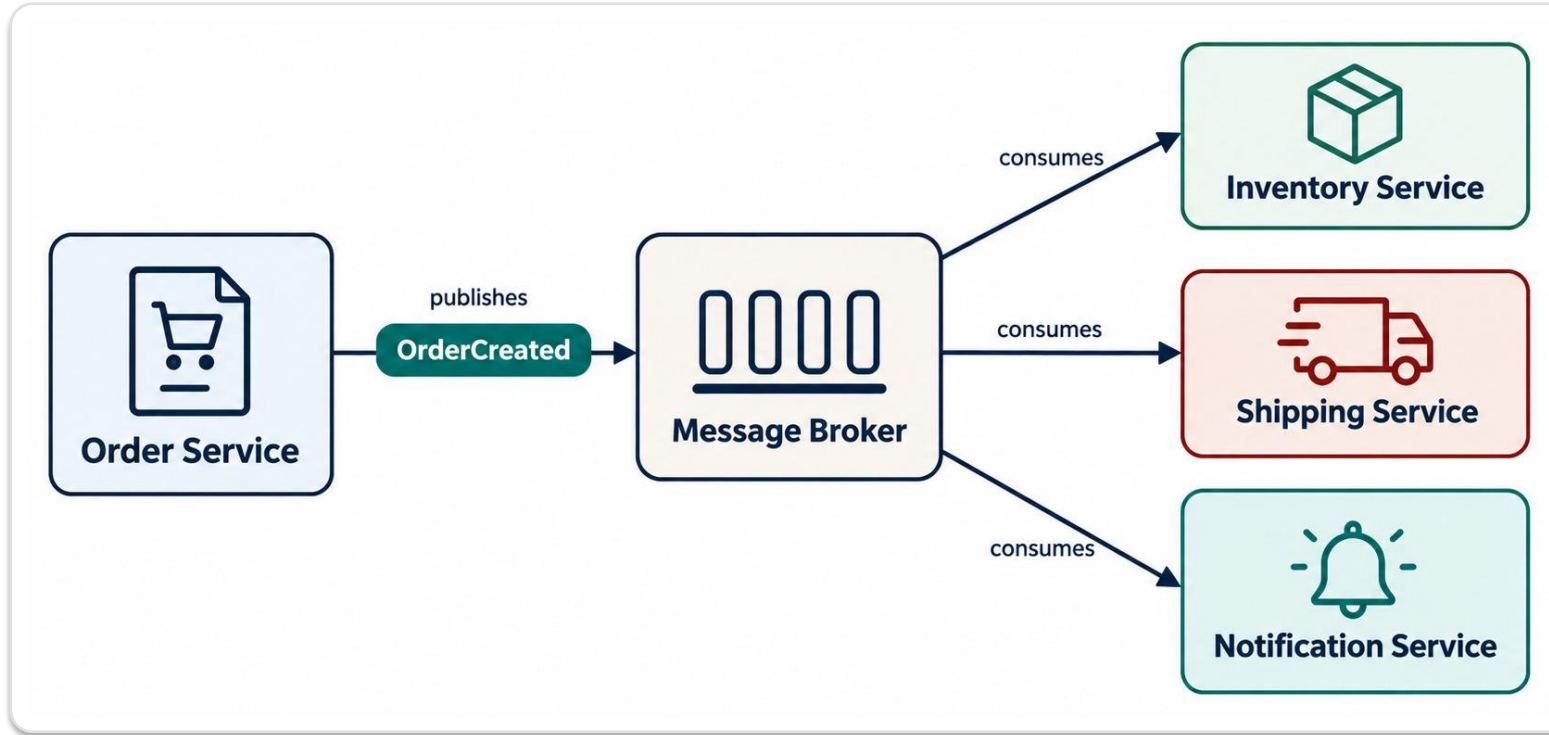
- ✗ Temporal coupling
- ✗ Latency adds up
- ✗ Failures propagate

ASK THE ROOM What happens to Order if Payment takes 30 seconds?

KEY TAKEAWAY The caller waits — so its latency and availability depend on every service downstream.

ASYNCHRONOUS MESSAGING AND EVENTS

Event-driven communication through brokers such as Kafka or RabbitMQ.



Command

Asks for something to happen:
ReserveInventory.

Event

States a fact, in past tense: OrderCreated.

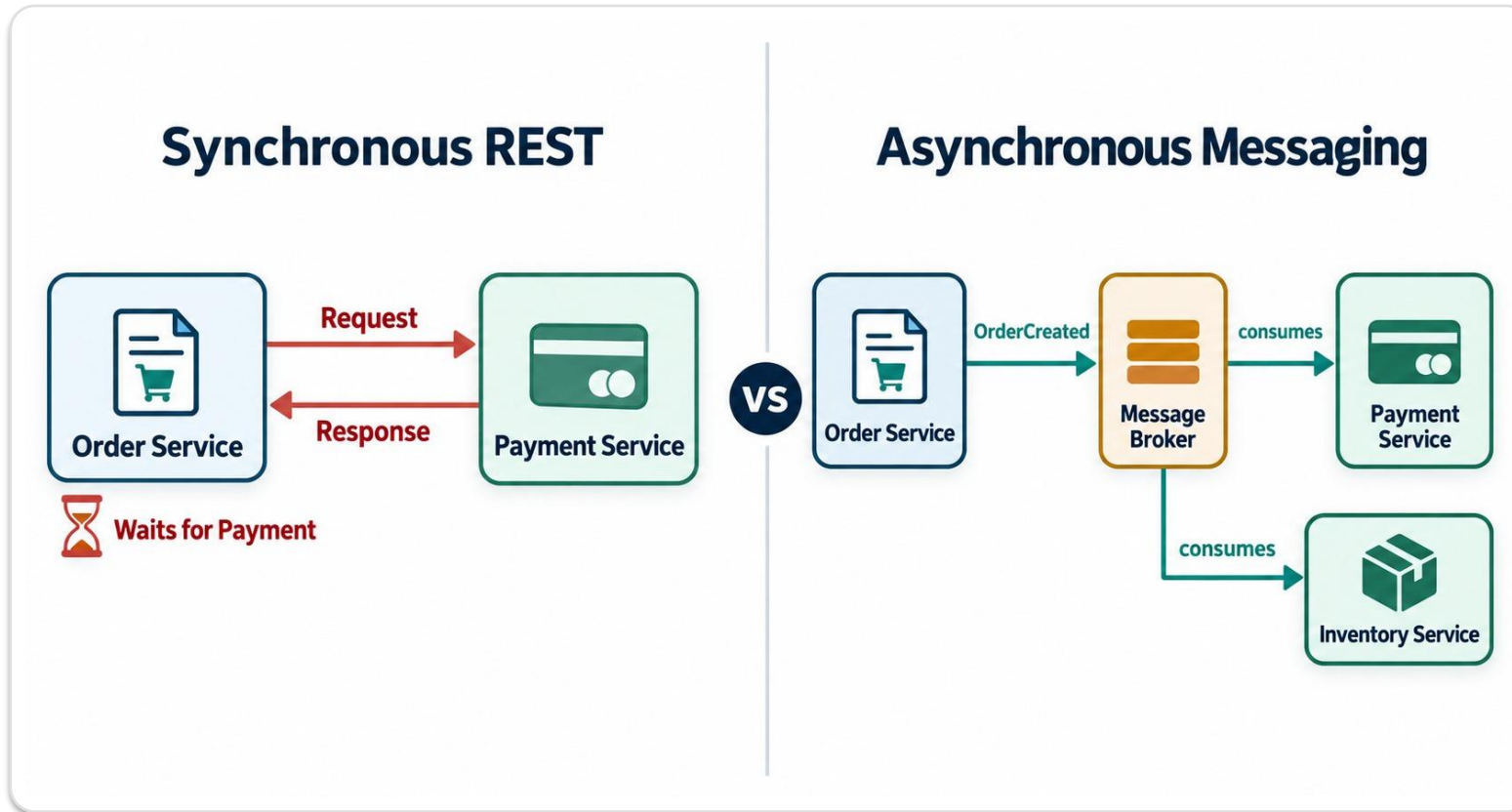
CONSUMERS MUST

- ✓ Handle duplicate messages safely
- ✓ Send repeated failures to a dead-letter queue

KEY TAKEAWAY The producer publishes and moves on — consumers react independently.

API VS. MESSAGING INTEGRATION PATTERNS

When request/response fits — and when events fit better.



Use REST when

The caller needs an answer now — e.g. GET /products/123.

Use messaging when

Work can happen later or many services react — e.g. OrderCreated.

NOT A COMPETITION Most real systems use both.

KEY TAKEAWAY Choose request/response for immediate answers and events for independent work.

A BOUNDED CALL AND A PUBLISHED EVENT

One order flow: ask Inventory now, tell everyone else later.

Synchronous: ask Inventory before accepting

```
Availability stock = inventory.get()
    .uri("/inventory/{sku}", sku)
    .retrieve()
    .body(Availability.class);

if (!stock.inStock()) {
    throw new OutOfStockException(sku);
}
// RestClient built with a timeout (Module 1)
```

Sync

Order needs stock availability before it accepts the order.

✓ Asynchronous: publish OrderCreated

```
@Service
public class OrderService {
    private final StreamBridge bridge;

    public OrderResponse place(Order order) {
        orders.save(order);
        bridge.send("orderCreated-out-0",
            new OrderCreated(order.id(),
                order.customerId(), order.total()));
        return OrderResponse.from(order);
    }
}
```

Async

Payment, Notification and Analytics react to OrderCreated.

Event payload

Carry IDs and facts — not the whole internal entity.

KEY TAKEAWAY Ask synchronously only when you need the answer now; publish events for everything that can happen later.

EXERCISE: REST OR MESSAGING?

Choose the communication style for each step of the order flow.

PRACTICE EXERCISE

10 MIN

SCENARIO

1. Check stock before accepting an order
2. Send the order confirmation email
3. Record an audit entry after payment
4. Show the current order status on screen
5. Tell Analytics an order was placed
6. Get a fraud score before charging the card

YOUR TASKS

- 1 Choose REST or an event for each step
- 2 Give the reason in a few words
- 3 Name one failure each choice must handle

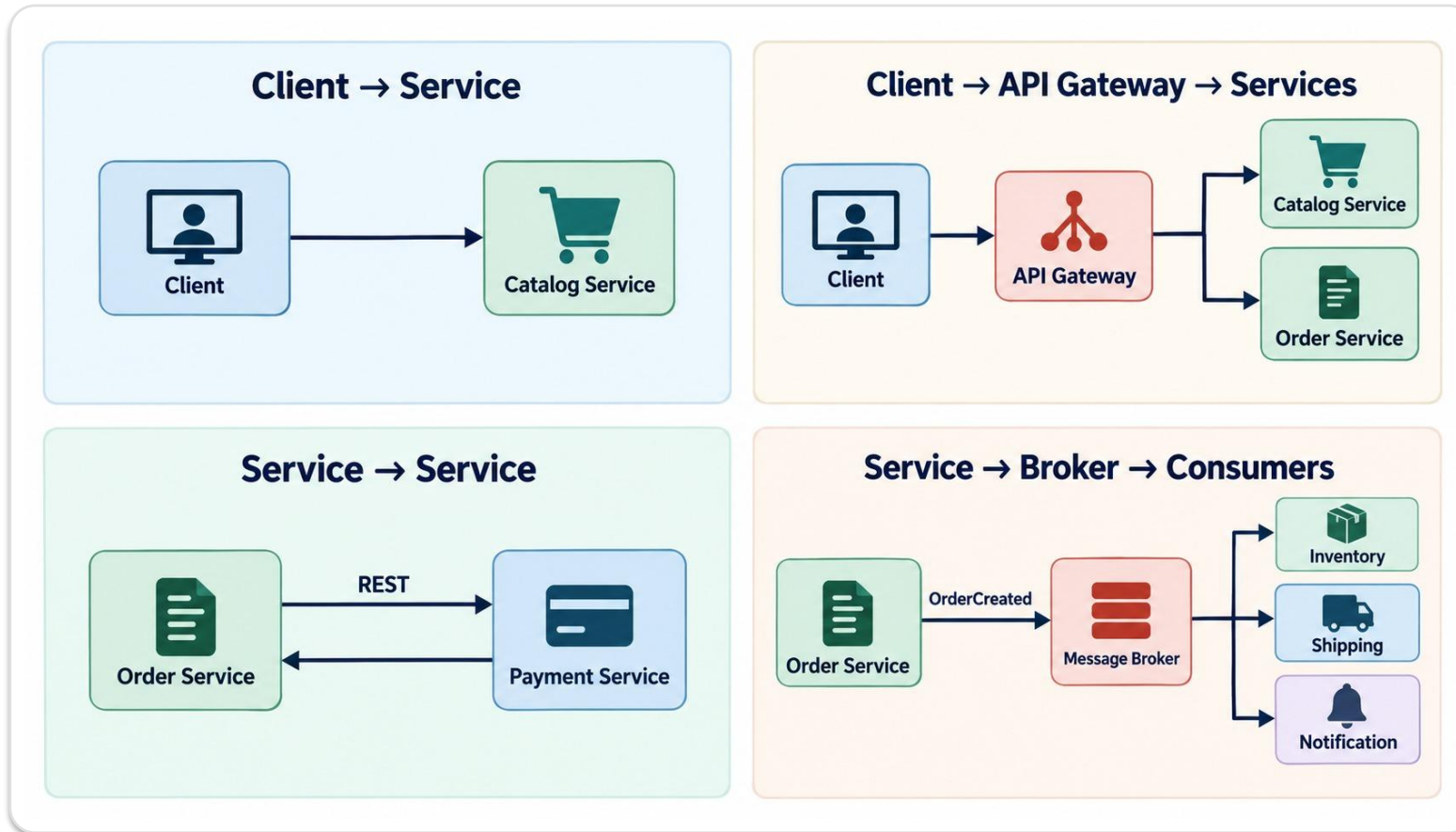
EXPECTED OUTCOME

- ✓ 1, 4, 6 → REST: an answer is needed now
- ✓ 2, 3, 5 → Event: work can happen later
- ✓ REST must handle timeouts and fallbacks
- ✓ Events must handle duplicates and dead letters

KEY TAKEAWAY Immediate decision → REST; background reaction or many listeners → messaging.

SERVICE COMMUNICATION STRATEGIES

Direct calls, API gateway, messaging and event-driven interactions.



Direct

Simple, but creates a direct dependency.

API gateway

One controlled entry point for web and mobile.

Service to service

Fits internal calls — adds runtime dependencies.

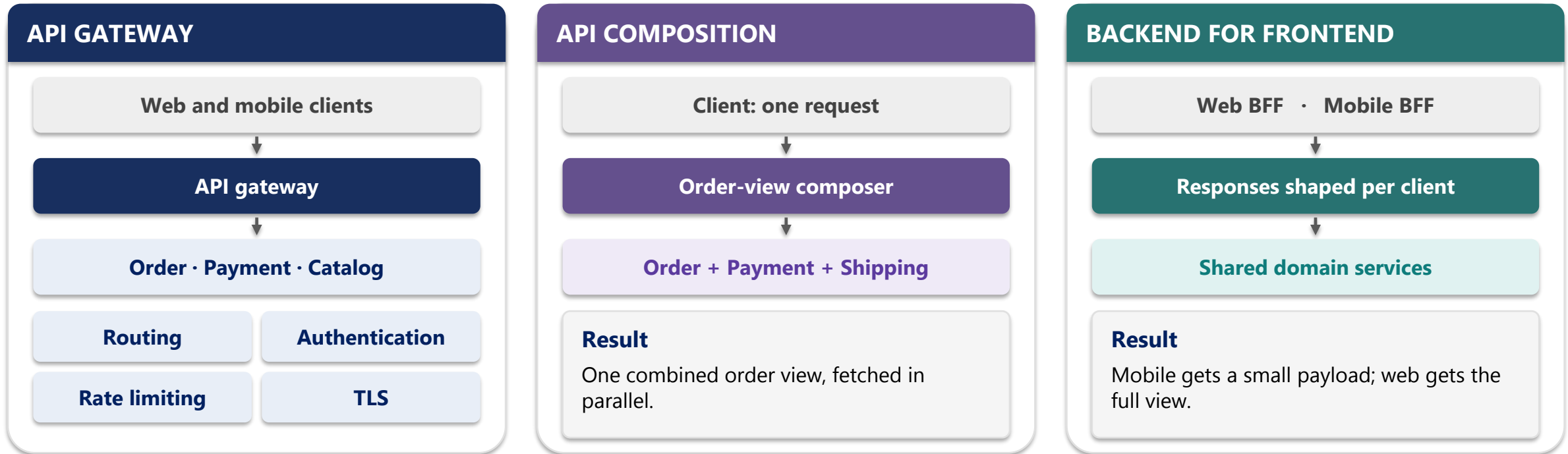
Messaging and events

Many consumers react without the producer knowing them.

KEY TAKEAWAY There's no universal strategy — deliberately choose one for each interaction.

API GATEWAY, COMPOSITION AND BFF

Keep clients simple — and stop them calling every service directly.



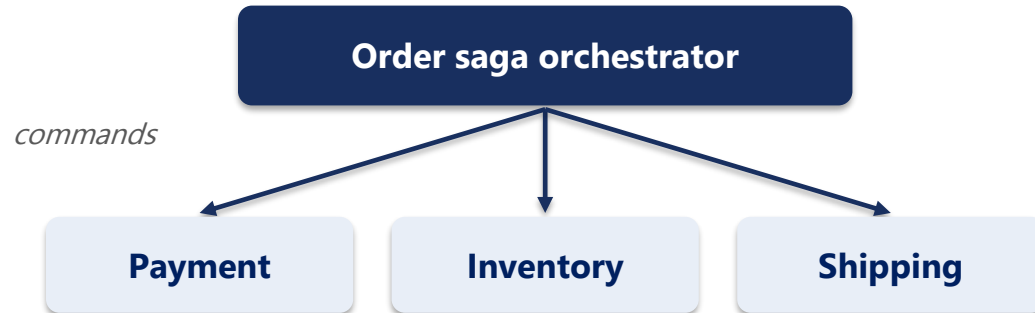
SERVICE DISCOVERY Callers use a logical name such as `http://payment-service` — OpenShift routes the call to a healthy pod.

KEY TAKEAWAY Gateways, composition and BFFs give each client one simple, well-shaped entry point.

ORCHESTRATION VS. CHOREOGRAPHY

Two ways to coordinate a workflow that spans several services.

ORCHESTRATION — CENTRAL CONTROL



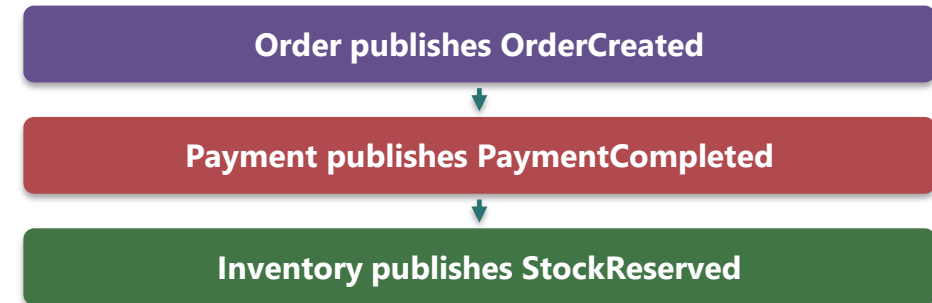
STRENGTHS

- ✓ The whole flow is visible in one place
- ✓ Easier to monitor and debug

WATCH OUT

- ✗ The orchestrator can become a bottleneck

CHOREOGRAPHY — SERVICES REACT



STRENGTHS

- ✓ No central coordinator
- ✓ Services evolve independently

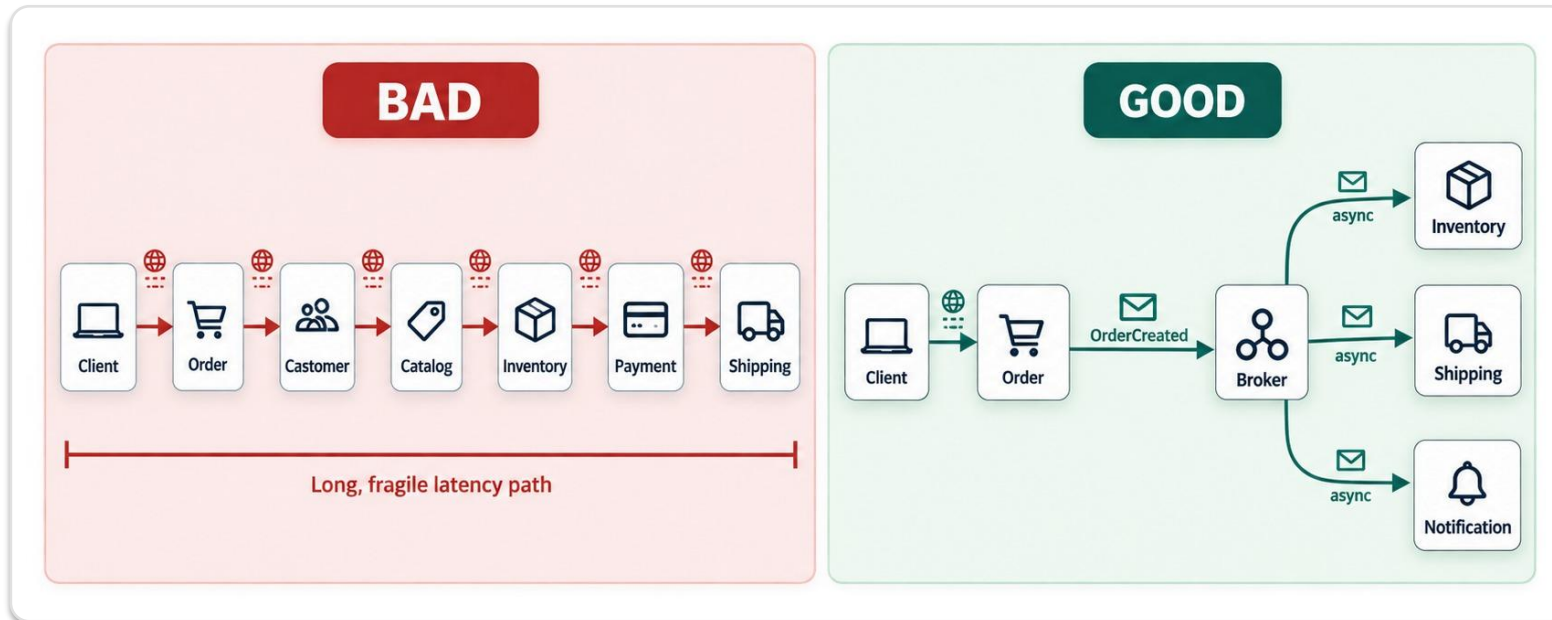
WATCH OUT

- ✗ The end-to-end flow is harder to trace

KEY TAKEAWAY Orchestration gives control and visibility; choreography gives independence — choose per workflow.

AVOIDING CHATTY SERVICE COMMUNICATION

Reduce excessive service-to-service calls and dependency chains.



EVERY NETWORK CALL ADDS

- ✗ Latency
- ✗ Serialization and authentication
- ✗ Another chance to fail

Better

Fewer, coarse-grained calls, plus events and well-drawn boundaries.

WHY In one JVM 20 calls are cheap; over a network they hurt.

KEY TAKEAWAY Don't design service APIs like local method calls — every network hop has a cost.

PART 4: WHERE STATE LIVES

The last design question before the blueprint.



SO FAR

- ✓ Choose REST when an answer is needed now
- ✓ Publish events for work that can happen later
- ✓ Keep calls coarse, bounded and few

NEXT QUESTION

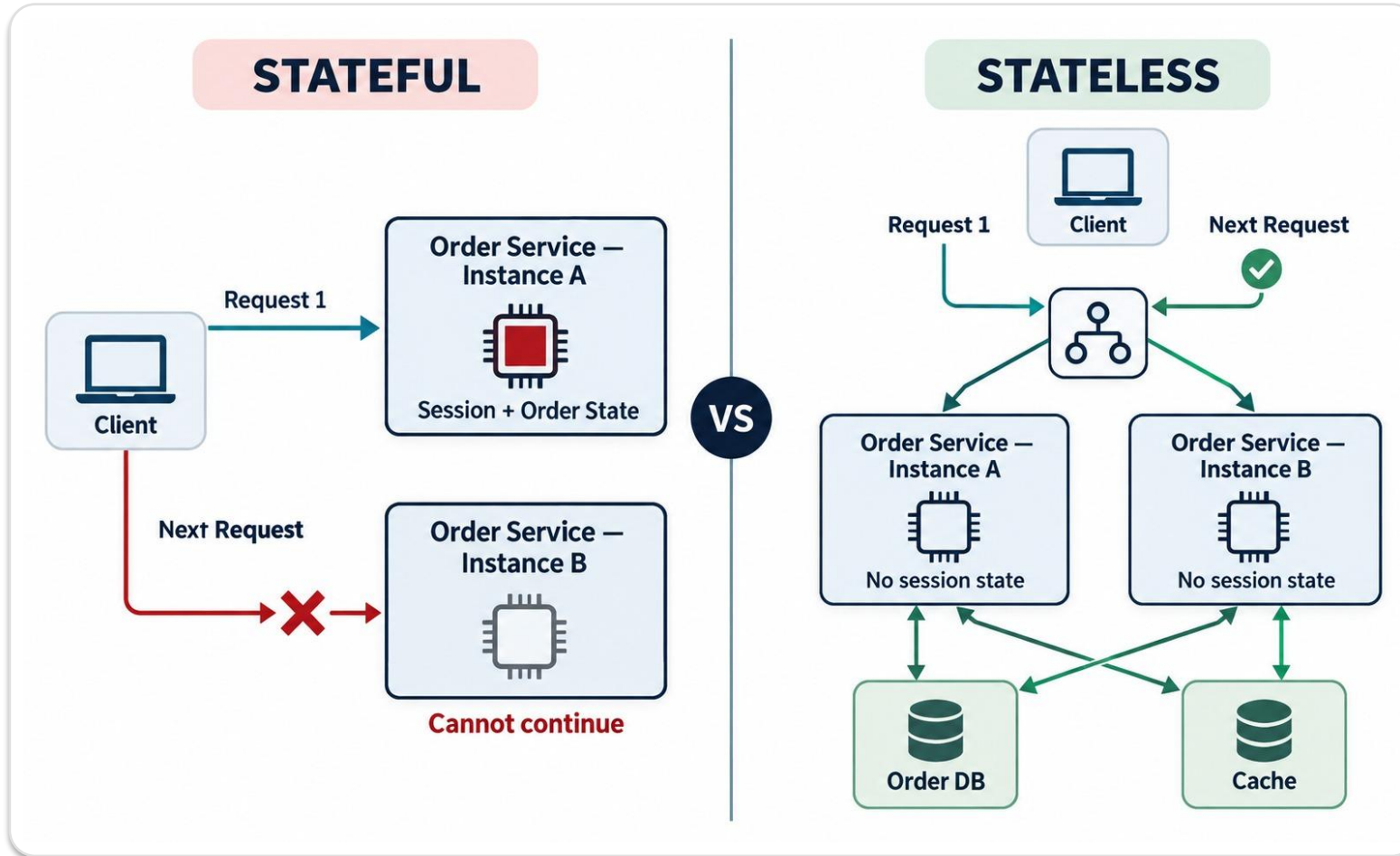
Where should state live, so any instance can serve any request?

IN THIS PART

- 1 Stateless versus stateful services
- 2 Scaling stateless instances
- 3 Stateless design in code — with an exercise
- 4 The complete design blueprint

STATELESS VS. STATEFUL SERVICES

Processing state versus persistent business state.



Stateless instance

Keeps no session state between requests — any pod can serve.

Stateful component

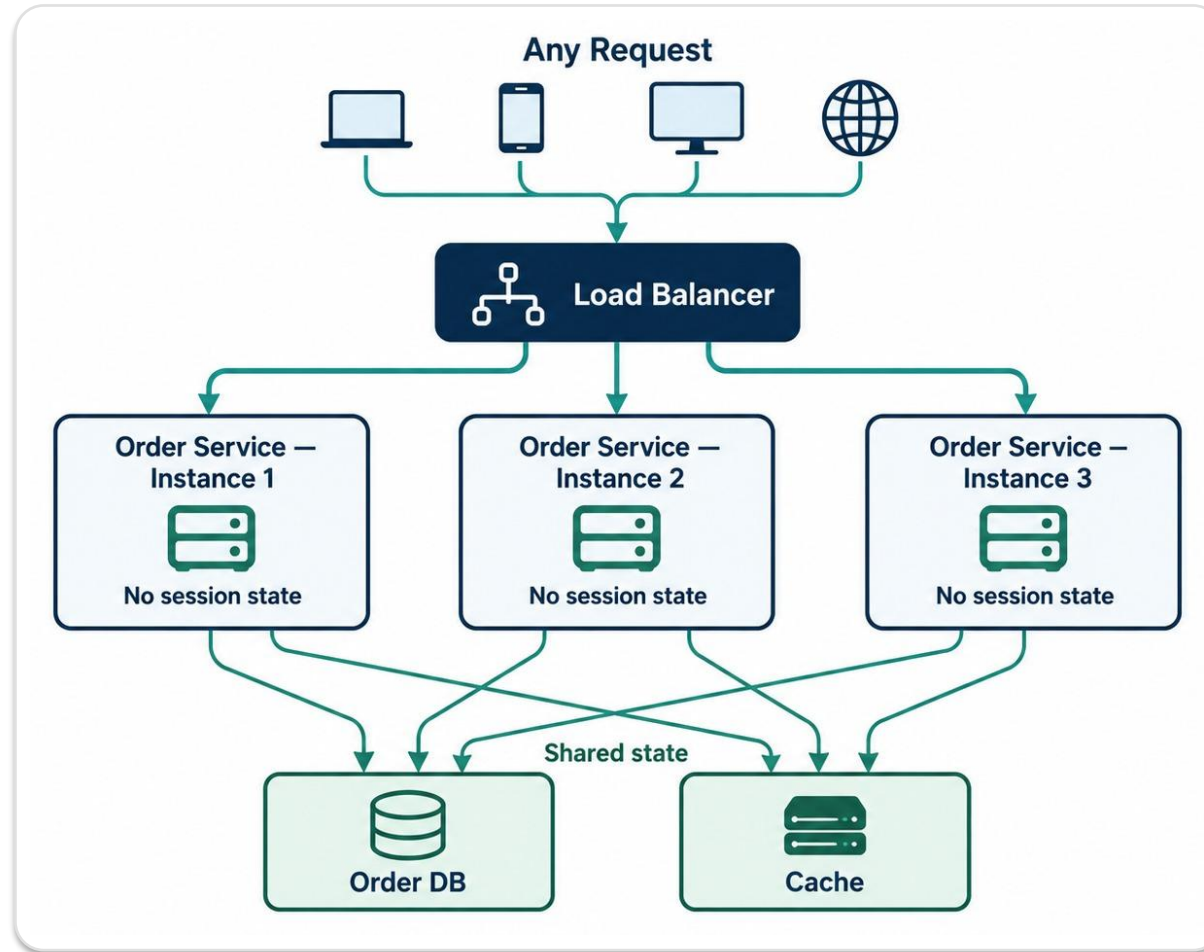
Databases, brokers and caches hold durable state.

STATELESS $\neq$ NO DATABASE The question is where state lives.

KEY TAKEAWAY A stateless instance doesn't rely on locally kept session state — durable state lives outside it.

DESIGNING STATELESS MICROSERVICES

Externalized state, horizontal scaling and container-friendly design.



WHY EXTERNALIZE STATE

- ✓ Requests spread across pods
- ✓ Pods restart and move
- ✓ Scaling and recovery stay easy

Where state goes

PostgreSQL for durable data, a shared cache for sessions, tokens for identity.

CACHES ARE FINE Correctness must not depend on one instance.

KEY TAKEAWAY Keep instances replaceable — so OpenShift can scale and recover Order Service freely.

STATELESS DESIGN IN CODE

Keep durable state out of the pod's memory.

✗ Lost on restart — different in every pod

```
@Service
public class CartService {
    private final Map<String, Cart> carts =
        new HashMap<>();

    public void addItem(String cartId, Item item) {
        carts.computeIfAbsent(cartId, Cart::new)
            .add(item);
    }
}
```

Bad

A restart loses the carts, and pod B can't see pod A's map.

✓ State lives in a backing service

```
@Service
public class CartService {
    private final CartRepository carts;

    public void addItem(String cartId, Item item) {
        Cart cart = carts.findById(cartId)
            .orElseGet(() -> new Cart(cartId));
        cart.add(item);
        carts.save(cart); // database or cache
    }
}
```

Good

Every pod uses the same store, so any pod can serve any request.

Identity

Carry who the user is in a token, not in server session memory.

KEY TAKEAWAY Application pods stay stateless; durable state belongs in databases, caches and tokens.

EXERCISE: STATELESS OR STATEFUL?

Classify the parts of the order platform.

PRACTICE EXERCISE

8 MIN

SCENARIO

- Order Service pod
- PostgreSQL order database
- Kafka broker
- Shared cart cache
- In-memory HashMap of pending payments
- Fraud-scoring REST API

YOUR TASKS

- 1 Label each one stateless or stateful
- 2 Which scale horizontally with no extra work?
- 3 Which is risky inside a container — and why?

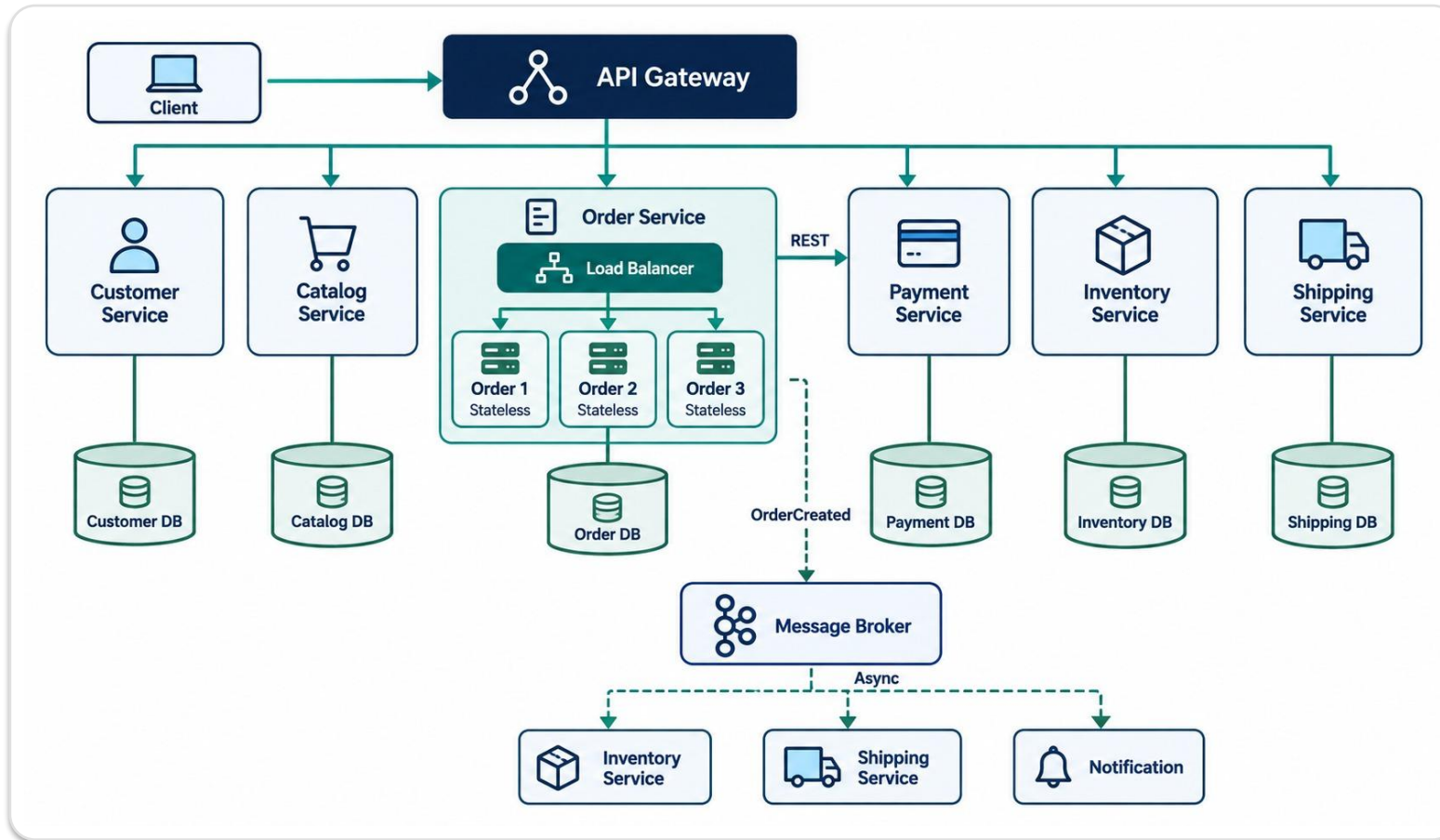
EXPECTED OUTCOME

- ✓ Stateless: Order Service pod, fraud-scoring API
- ✓ Stateful: PostgreSQL, Kafka, cart cache
- ✓ Risky: the in-memory HashMap — lost on restart
- ✓ Fix: store pending payments in the database

KEY TAKEAWAY Scale the stateless tier freely; protect and back up the stateful components.

PUTTING IT TOGETHER — MICROSERVICES DESIGN BLUEPRINT

Boundaries, APIs, messaging, data ownership and stateless instances.



THE DESIGN SEQUENCE

- ✓ Business domain
- ✓ Bounded context
- ✓ Service boundary
- ✓ Cohesion and loose coupling
- ✓ API contract
- ✓ Communication strategy
- ✓ Data and state ownership

EVERY BOX comes from a part of this module.

KEY TAKEAWAY Good microservices design starts with business boundaries — not REST controllers.

KNOWLEDGE CHECK

Answer in your own words — then compare with a partner.

1 What is a REST resource?

2 Why should URIs use nouns instead of verbs?

3 Which HTTP method usually creates a resource?

4 Which status code means a resource was created?

5 Why should APIs use DTOs?

6 What is loose coupling?

7 What is high cohesion?

8 What is a bounded context?

9 When should messaging be used instead of REST?

10 What does an OpenAPI document describe?

EXERCISE 1.2 — DESIGN ACCOUNT SERVICE APIS AND BOUNDARIES

Module 2 checkpoint B: REST resources, status transitions and an OpenAPI sketch.

OFFICIAL CHECKPOINT B

15 MIN

curriculum/day-01/exercises/exercise-1.2-account-apis.md

SCENARIO

In Lab 1 you build the banking Account Service. Before coding, sketch its API.

Resource: an account. Status machine:
PENDING → ACTIVE → FROZEN or CLOSED.

YOUR TASKS

- 1 Sketch POST, GET and PATCH plus activate, freeze and close — no DELETE
- 2 Model the statuses: freezing a PENDING account is 409; FROZEN can be activated; retries return 200
- 3 List the 400, 404 and 409 cases; note OpenAPI and Actuator as Lab 1 deliverables
- 4 Draw the bounded context: Account owns accounts only — not transactions or risk scores

EXPECTED OUTCOME

- ✓ A one-page Account API sketch
- ✓ Lab 1 can implement it without changing URLs
- ✓ Use this contract in Lab 1
- ✓ No Customer or Notification services

KEY TAKEAWAY A one-page Account API sketch that Lab 1 can implement without changing URLs tomorrow.

MODULE 2 SUMMARY AND WHAT'S NEXT

From boundaries to a complete design blueprint.

YOU CAN NOW

- ✓ Find boundaries from capabilities and bounded contexts
- ✓ Keep coupling low and cohesion high
- ✓ Design resource-oriented REST APIs with stable contracts
- ✓ Choose REST or messaging for each interaction
- ✓ Keep services stateless and state in backing services

NEXT: MODULE 3 — SPRING BOOT DEVELOPMENT

- 1 How do we build the Account Service in Spring Boot?
- 2 How are controllers, services and repositories organized?
- 3 How do we configure, validate and document it?

BRING ALONG Your Exercise 1.2 Account API sketch — Lab 1 implements it.

KEY TAKEAWAY Module 3 turns this design into a working Spring Boot service.